



Vulnerability Assessment & Penetration Testing Report

Target:

<https://ts.offenso.local>

Prepared For:

Hackware Security Solutions LLP

Prepared By:

Basith Mohammed Ali

Date:

9 July 2026

Version:

1.0

Table of Contents

1. Executive Summary	3
2. Scope	3
3. Methodology	3
4. Findings Summary	4
F-01 – Network Enumeration	5
F-02 – Directory and Endpoint Enumeration	6
F-03 – Account Takeover via Weak JWT Secret Key	7
F-04 – Negative Price / Quantity Manipulation	10
F-05 – OTP and Sensitive Information Exposure in Client-Side Cookie	12
F-06 – Insecure Direct Object Reference (IDOR) in Order Details	14
F-07 – Session Hijacking via Insecure HTTP Transmission	17
F-08 – Reflected Cross-Site Scripting (XSS)	20
Conclusion	22

1. Executive Summary

A Vulnerability Assessment and Penetration Testing (VAPT) engagement was conducted on the Tokyo Store web application (<https://ts.offenso.local>) to identify security weaknesses that could impact the confidentiality, integrity, and availability of the system. Multiple vulnerabilities were identified, including authentication, authorization, session management, business logic, and input validation issues. The findings demonstrate the potential for unauthorized account access, sensitive information disclosure, session hijacking, and client-side attacks. Detailed technical findings and remediation recommendations are provided in this report.

2. Scope

The assessment was performed against the Tokyo Store web application hosted at:

Target	https://ts.offenso.local
--------	---

The testing focused on application functionality, authentication mechanisms, authorization controls, session management, input validation, and business logic security.

3. Methodology

Phase	Description
Reconnaissance & Enumeration	Identification of exposed services and endpoints
Vulnerability Identification	Discovery of security weaknesses
Vulnerability Validation	Manual verification and testing
Risk Assessment	Impact analysis
Reporting	Documentation and recommendations

4. Findings Summary

ID	Finding	Severity
F-01	Network Enumeration (Nmap)	Informational
F-02	Directory and Endpoint Enumeration	Informational
F-03	Account Takeover via Weak JWT Secret Key	High
F-04	Negative Price / Quantity Manipulation	Medium
F-05	OTP and Sensitive Information Exposure in Client-Side Cookie	Medium
F-06	Insecure Direct Object Reference (IDOR)	Medium
F-07	Session Hijacking via Insecure HTTP Transmission	Critical
F-08	Reflected Cross-Site Scripting (XSS)	Medium

Severity	Count
Critical	1
High	1
Medium	4
Low	0
Informational	2

The assessment identified one Critical, one High, four Medium, and two Informational findings. Due to the presence of a Critical vulnerability affecting

account security, the overall risk rating is Critical. security and multiple High-risk vulnerabilities impacting authorization, session management, and business logic, the overall risk rating is Critical.

F-01 – Network Enumeration

Severity	Informational
CVSS v3.1	N/A
CVE	N/A
Protocol	TCP
Ports	22 (SSH), 80 (HTTP), 443 (HTTPS)

Description

A network enumeration scan was performed against the target host **192.168.56.114** using Nmap. The scan identified three accessible services: SSH (22), HTTP (80), and HTTPS (443). Service versions and operating system information were successfully obtained from the target.

Proof of Concept

Command Used

`nmap -A -T4 192.168.56.114`

Results

- SSH: OpenSSH 9.6p1 Ubuntu
- HTTP: Nginx 1.24.0
- HTTPS: Nginx 1.24.0
- Operating System: Linux (Ubuntu)

```

L-$ nmap -A -T4 192.168.56.114
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-12 08:14 +0530
Nmap scan report for ts.offenso.local (192.168.56.114)
Host is up (0.029s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|_ 256 79:6b:81:15:45:cc:91:5c:74:1d:67:38:b2:76:3a:36 (ECDSA)
|_ 256 16:a1:41:2e:93:a5:f2:e9:21:72:f3:b7:fc:09:d2:b9 (ED25519) Used
80/tcp    open  http     nginx 1.24.0 (Ubuntu)
|_ http-title: Did not follow redirect to https://ts.offenso.local/
443/tcp   open  ssl/http nginx 1.24.0 (Ubuntu)
|_ tls-alpn:
|_ h2
|_ http/1.1
|_ http/1.0
|_ http/0.9
|_ ssl-cert: Subject: commonName=*.offenso.local/organizationName=Offenso Security/stateOrProvinceName=Kerala/countryName=IN
|_ Subject Alternative Name: DNS:*.offenso.local, DNS:offenso.local
Results
* SSH: OpenSSH 9.6p1 Ubuntu
* HTTP: Nginx 1.24.0
|_ ssl-cert: Subject: commonName=*.offenso.local/organizationName=Offenso Security/stateOrProvinceName=Kerala/countryName=IN
|_ Subject Alternative Name: DNS:*.offenso.local, DNS:offenso.local
```

Impact

The exposed services provide information that can be used during further security testing and may assist an attacker in identifying potential attack vectors.

Recommendation

Limit exposure of unnecessary services, keep software updated, and restrict access to administrative services such as SSH where possible.

Remediation Status

Open

F-02 – Directory and Endpoint Enumeration

Severity	Informational
CVSS v3.1	N/A

Description

Directory enumeration was performed using Gobuster against the target application. Multiple accessible endpoints were identified, including authentication, administration, shopping cart, product, and user management functionality.

Proof of Concept

Command Used

```
gobuster dir -u https://ts.offenso.local/ -w /usr/share/wordlists/dirb/common.txt -t 50 -k
```

/admin	/cart	/category	/checkout	/dashboar d	/logout	/login
/orders	/product	/register	/search	/subscribe	/upload	

```

=====
Starting gobuster in directory enumeration mode
=====
admin      (Status: 302) [Size: 199] [--> /login]
cart       (Status: 302) [Size: 199] [--> /login]
category   (Status: 200) [Size: 13415]
change_password (Status: 405) [Size: 153]
checkout   (Status: 405) [Size: 153]
dashboard  (Status: 302) [Size: 199] [--> /login]
login      (Status: 200) [Size: 14467]
logout     (Status: 302) [Size: 189] [--> /]
orders     (Status: 302) [Size: 199] [--> /login]
product    (Status: 200) [Size: 17953]
register    (Status: 200) [Size: 15556]
search     (Status: 302) [Size: 203] [--> /product]
subscribe  (Status: 400) [Size: 50]
upload     (Status: 302) [Size: 199] [--> /login]
Progress: 4613 / 4613 (100.00%)
=====
Finished

```

Impact

The discovered endpoints provide valuable information about the application's structure and functionality, which may assist further security testing.

Recommendation

Restrict access to sensitive administrative functionality and ensure unnecessary endpoints are removed or properly protected.

F-03 – Account Takeover via Weak JWT Secret Key

Severity	High
CVSS v3.1	8.8
Affected Endpoint	https://ts.offenso.local/dashboard

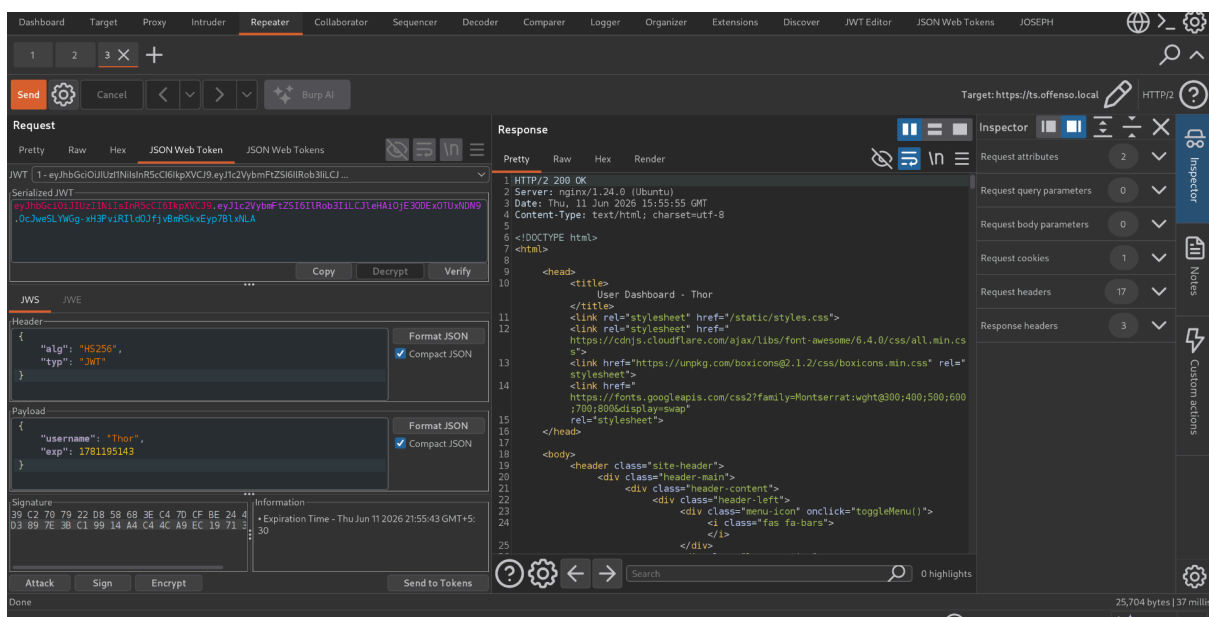
Description

The application uses JWT tokens for authentication. During testing, the JWT signing secret was discovered to be weak and predictable (`jwt_secret`). An attacker can forge valid JWT tokens and impersonate other users without knowing their passwords.

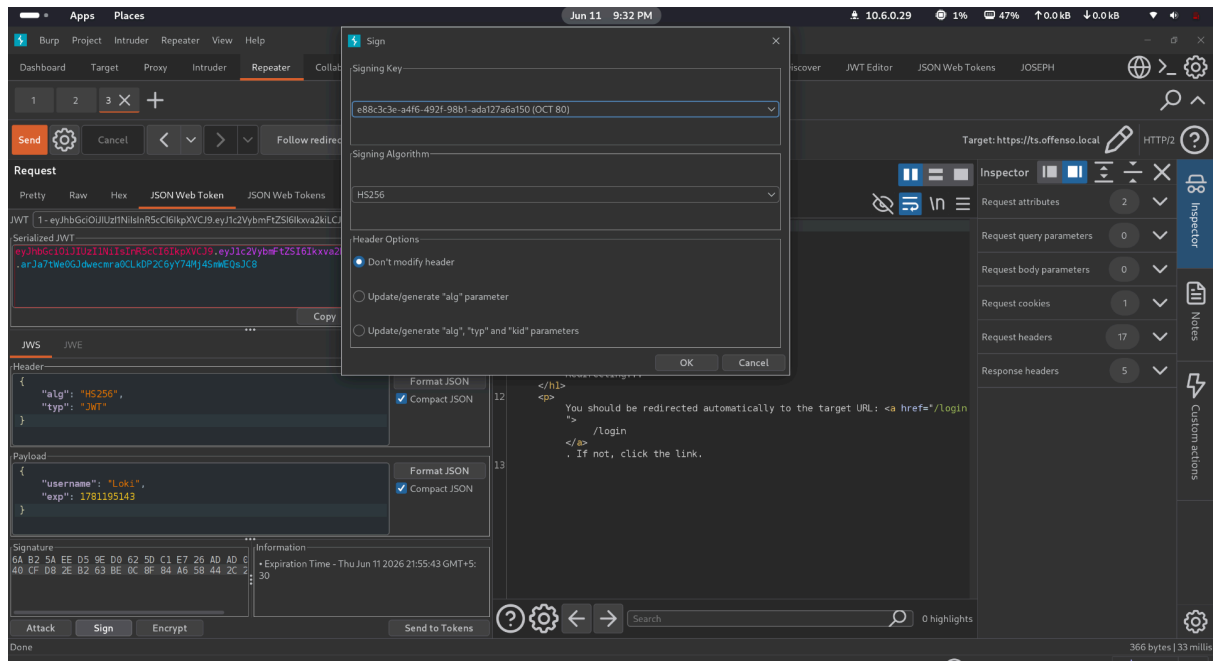
Proof of Concept

1. A valid JWT token was captured from the application.
2. The JWT signing secret was identified as "**jwt_secret**".
3. A forged JWT token was generated by modifying the username from **Thor** to **Loki** and extending the token expiration time.
4. The modified JWT token was accepted by the application without additional verification.
5. Successful access to the **Loki** user account was obtained using the forged JWT token.

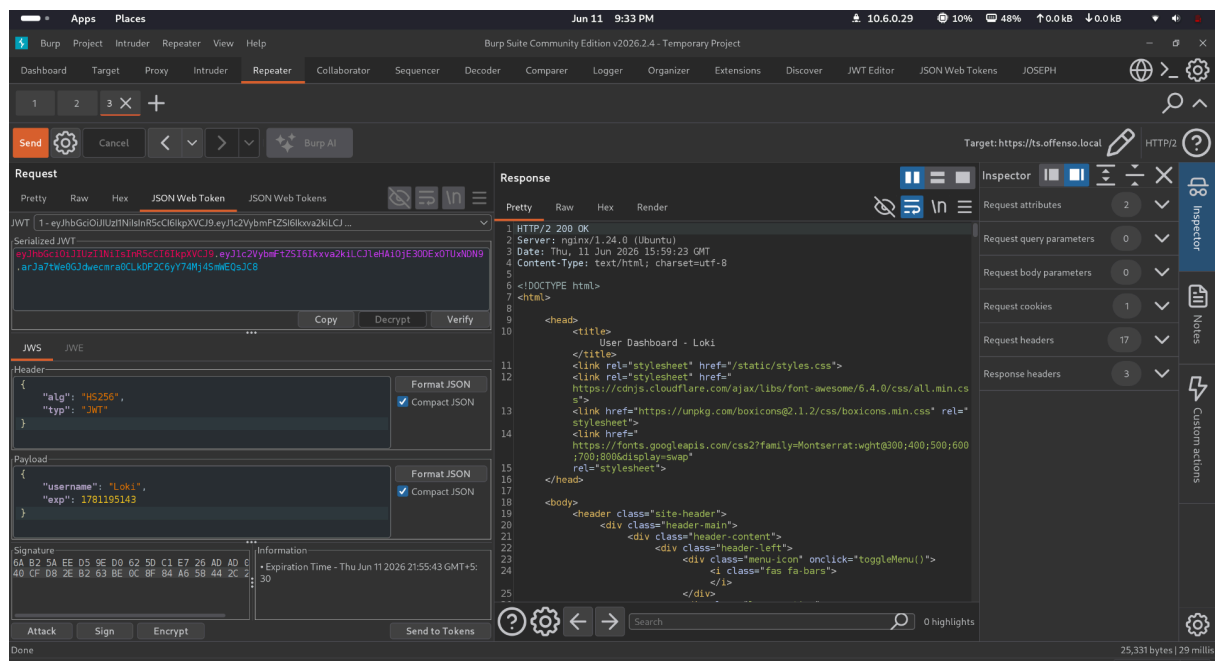
Original JWT token associated with user Thor.



*JWT token signed using the discovered secret key and modified from user **Thor** to user **Loki**.*



*Successful account access as user **Loki** using the forged JWT token.*



Impact

An attacker can impersonate arbitrary users, gain unauthorized access to accounts, modify user data, and potentially compromise the entire application.

Recommendation

- Use a strong, randomly generated JWT secret.
- Rotate existing JWT secrets.
- Validate token claims securely.
- Implement short token lifetimes.
- Consider asymmetric signing algorithms (RS256).

F-04 – Negative Price / Quantity Manipulation

Severity	Medium
CVSS v3.1	6.5
Affected Endpoint	https://ts.offenso.local/add-to-cart/{product_id}

Description

The application fails to validate quantity values supplied by the client. By modifying the quantity parameter to a negative value, it was possible to manipulate pricing calculations and create orders with negative totals.

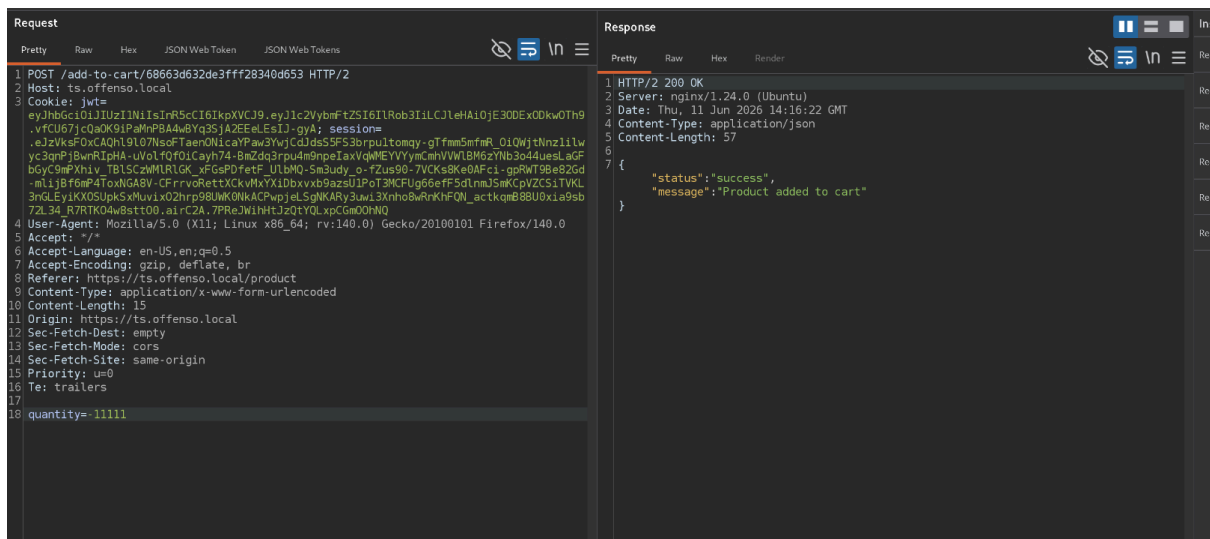
Proof of Concept

1. A product was added to the shopping cart.
2. The quantity parameter in the add-to-cart request was modified to:

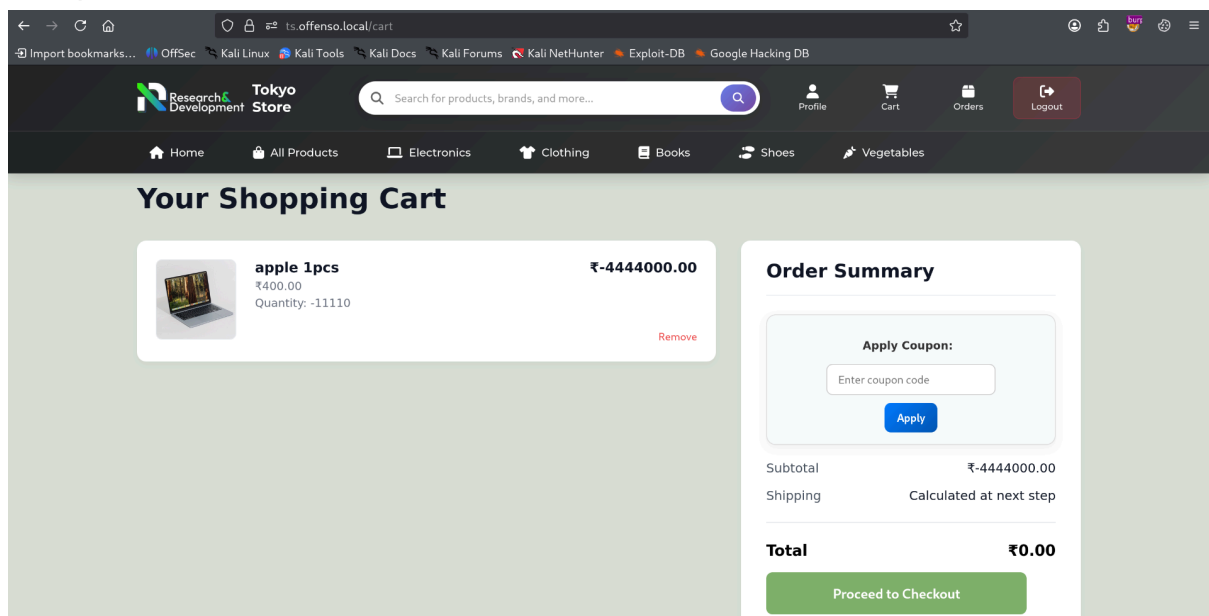
quantity=-11111

3. The application accepted the modified request.
4. The shopping cart displayed a negative quantity and negative subtotal.
5. The checkout process continued successfully.
6. An order was generated with a negative total amount.

Modified add-to-cart request containing a negative quantity value



Shopping cart displaying a negative quantity and negative subtotal after manipulation.



Impact

An attacker can manipulate product quantities and pricing calculations, resulting in invalid orders, financial loss, business logic abuse, and unauthorized transactions.

Recommendation

Implement strict server-side validation for quantity values. Reject negative, zero, and unreasonable quantity values before performing any cart, checkout, payment, or order calculations.

F-05 – OTP and Sensitive Information Exposure in Client-Side Cookie

Severity	Medium
CVSS v3.1	6.5
Affected Endpoint	https://ts.offenso.local/verify-otp

Description

The application stores sensitive payment and order information within a client-side session cookie during the payment verification process. After entering card details and proceeding to the OTP verification page, the session cookie contained the OTP value along with order details, delivery address information, product information, and username data.

By decoding the session cookie, sensitive information intended for server-side processing was disclosed to the client.

Proof of Concept

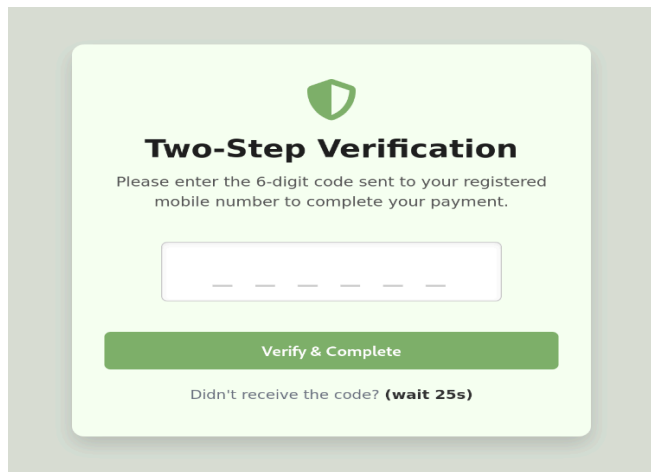
1. A product was added to the cart and the checkout process was initiated.
2. Card details were entered and the application redirected to the OTP verification page.
3. The session cookie was captured from the request using Burp Suite.
4. The cookie value was decoded using a Python script.
5. The decoded data revealed sensitive information including:
 - Username
 - OTP value
 - Product details
 - Order details
 - Delivery address information
 - Payment workflow data

Extract from the decoded session cookie:

```
{
  "username": "Thor",
  "otp": "009664",
  "product_name": "Iphone 16",
  "total_price": 236578.0,
  "delivery_address": {
    "city": "kannur",
    "state": "kerala",
    "zip_code": "670301"
  }
}
```

6.The exposed OTP was used during verification and the order was successfully placed.

OTP verification page after card payment submission.



Session cookie captured from the OTP verification request

```
Request
Pretty Raw Hex JSON Web Token JSON Web Tokens
1 GET /verify-otp HTTP/2
2 Host: ts.offenso.local
3 Cookie: jwt=
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImlRob3IiLCJleHAiOiJlE30DEx0TEwMTB9.-8A00LFUXsS78qDEL7GNL7w6GWrrZd3oB1Ph7xybQTA; session=
.eJzdUk1v3CAQ_SuIcIn5A7NWbk3UQ1VVG2LXvUqrNAGcRbGBAk6zXfm_B9haiFITysWeN_PeMG-4YKeM10
aJWY-VxzfogUPZqfSDBfiIvyA8B-UNTAU7nKzPmPNWziKGhD1c1ohrmWtYz1grWdtI1Q7D0PQtrSTr2g88v
uqBc6NCtRMhZ3_PYK055QhdTpVYWiRS2lVbXKso5p4t8HGXEXTqdbMBE9F9Pbb_kB-fr273e1-7E1T0d69
kpPylrSvDak3f9Sjw8sxUYo0X1u8qy25MZwnZSKXKoIeQ7HGRpcbNH1Duy7fuLjGPzv2n7gj1ahfLD9zkNK
rcLXg0gWFSgxd_0h6UTE1owRYNu0sJqKrt02oLU0331Lzn_SzqVnlrFRG1Vn8BmMmf0KNRnKgbj0-CEdIs
Q186w8jJChv9pxYwVB2bZqz7pQ7WyiL-zv6VkdXvIm-TBCOKl_60Axf_GUBsqOpPzucI9CWjUarEdpYD1oA
VFbs0F34CW69_ZFpz3foF86AD4ux-UNe8f0hg.airKhg.nyGhHbYHNgWqm9spMLJukNjp4Zo
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Referer: https://ts.offenso.local/card-payment
9 Sec-Fetch-Dest: empty
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Site: same-origin
12 Priority: u=0
13 Te: trailers
14
```

Decoded session cookie revealing OTP and sensitive order information

```
(venv)kali@kali: ~/Internship/TS/OTP-IDOR
--$ python3 OTP.py
{"pending_order": {"type": "cart", "username": "Thor", "products": [{"product_id": "68663d632de3fff28340d653", "product_name": "apple 1pcs", "quantity": -11110, "price": 400.0, "item_total": -4444000.0, "image": "BEST-MACBOOKS-2048px-hero-3x2-1.webp"}], "total_price": -4444000.0, "payment_details": {"otp": "282455", "order_type": "cart", "products": [{"product_id": "68663d632de3fff28340d653", "product_name": "apple 1pcs", "quantity": -11110, "price": 400.0, "item_total": -4444000.0, "image": "BEST-MACBOOKS-2048px-hero-3x2-1.webp"}], "delivery_address": {"id": "4a0cf58b-8c12-4166-a675-614c5372c3e4", "type": "Shipping", "line1": "kannur", "line2": "", "city": "kannur", "state": "kerala", "zip_code": "670301", "country": "india"}}, {"_flashes": [{"t": ["message", "OTP sent for verification. Card Provider: Visa"]}]}
```

Impact

An attacker with access to the user's session cookie could obtain sensitive payment workflow information, including OTP values, order details, and delivery address information. Storing OTPs and sensitive transaction data within client-accessible cookies increases the risk of information disclosure, account compromise, payment abuse, and unauthorized transaction completion.

Recommendation

- Store OTP values exclusively on the server side.
- Do not store sensitive payment or order information in client-side cookies.
- Use secure server-side session management.
- Minimize data stored within session cookies.
- Apply proper cookie security attributes such as **HttpOnly**, **Secure**, and **SameSite**.
- Encrypt and protect sensitive transaction data on the server.

F-06 – Insecure Direct Object Reference (IDOR) in Order Details

Severity	Medium
CVSS v3.1	5.3
Affected Endpoint	https://ts.offenso.local/orders/{order_id}

Description

The application fails to enforce proper authorization checks when accessing order details. Order records are referenced using a predictable order identifier in the URL. By modifying the order ID parameter, an authenticated user can access order information belonging to another user without authorization.

Proof of Concept

1. User **Loki** placed an order, generating the following order ID:

`6a2ad0dd53d680c22d1223ec`

While authenticated as **Thor**, the order ID in the URL was modified from Thor's order ID to Loki's order ID:

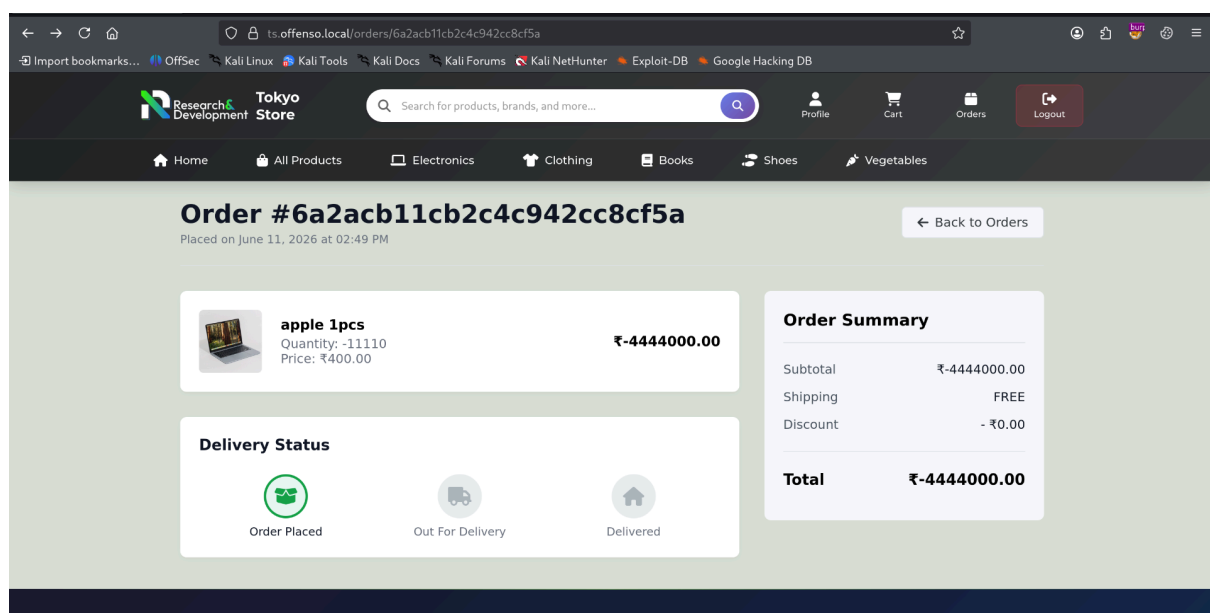
`/orders/6a2ad0dd53d680c22d1223ec`

The application returned Loki's order details without verifying ownership of the order.

Thor was able to view information belonging to Loki, including:

- Product details
- Order value
- Order status
- Order information

Thor's legitimate order details.



Loki's legitimate order details.

The screenshot shows a web browser at the URL `ts.offenso.local/orders/6a2ad0dd53d680c22d1223ec`. The page is for 'Tokyo Store' and displays order details for order #6a2ad0dd53d680c22d1223ec, placed on June 11, 2026, at 03:14 PM. The order contains one item, 'The onion', with a quantity of -11110 and a price of ₹222.00, totaling ₹-2466420.00. The delivery status is 'Order Placed'. The order summary shows a subtotal of ₹-2466420.00, free shipping, and a total of ₹-2466420.00.

Order #6a2ad0dd53d680c22d1223ec
Placed on June 11, 2026 at 03:14 PM

The onion
Quantity: -11110
Price: ₹222.00
₹-2466420.00

Delivery Status

- Order Placed
- Out For Delivery
- Delivered

Order Summary

Subtotal	₹-2466420.00
Shipping	FREE
Discount	- ₹0.00
Total	₹-2466420.00

Loki's order details successfully accessed from Thor's account by modifying the order ID.

This screenshot is identical to the one above, showing the same order details for 'Tokyo Store'. The browser's address bar and the page content are the same, indicating that the order information was successfully accessed from a different account (Thor's) by modifying the order ID.

Order #6a2ad0dd53d680c22d1223ec
Placed on June 11, 2026 at 03:14 PM

The onion
Quantity: -11110
Price: ₹222.00
₹-2466420.00

Delivery Status

- Order Placed
- Out For Delivery
- Delivered

Order Summary

Subtotal	₹-2466420.00
Shipping	FREE
Discount	- ₹0.00
Total	₹-2466420.00

Impact

An attacker can access order information belonging to other users by changing the order identifier. This may lead to unauthorized disclosure of sensitive business and customer information and could potentially allow further attacks against user accounts.

Recommendation

Implement server-side authorization checks for all order-related requests. The application should verify that the requested order belongs to the currently authenticated user before returning any order information. Direct object references should never be relied upon as the sole access control mechanism.

F-07 – Session Hijacking via Insecure HTTP Transmission

Severity	Critical
CVSS v3.1	9.1
CWE	CWE-319: Cleartext Transmission of Sensitive Information
Protocol	HTTP
Affected Endpoint	http://ts.offenso.local/dashboard

Description

The application allows authenticated requests over HTTP and transmits session-related cookies without the Secure attribute. During testing, the JWT session token was observed being transmitted in cleartext when accessing the application over HTTP.

An attacker positioned on the same network could capture the victim's session cookie and reuse it to impersonate the victim without requiring valid credentials.

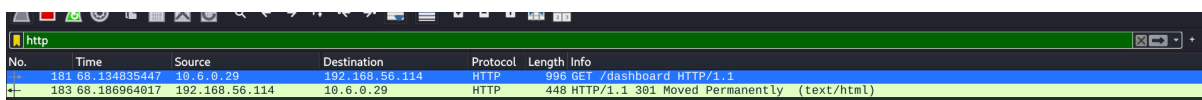
Proof of Concept

1. Capture Session Cookie

A request was sent to:

```
GET /dashboard HTTP/1.1
Host: ts.offenso.local
Cookie: jwt=<victim_jwt>
```

Using Wireshark, the JWT session token was observed in cleartext within the HTTP request.



No.	Time	Source	Destination	Protocol	Length	Info
181	68.134835447	10.6.0.29	192.168.56.114	HTTP	996	GET /dashboard HTTP/1.1
183	68.186964017	192.168.56.114	10.6.0.29	HTTP	448	HTTP/1.1 301 Moved Permanently (text/html)

```
Wireshark · Follow TCP Stream (tcp.stream eq 2) · tun0

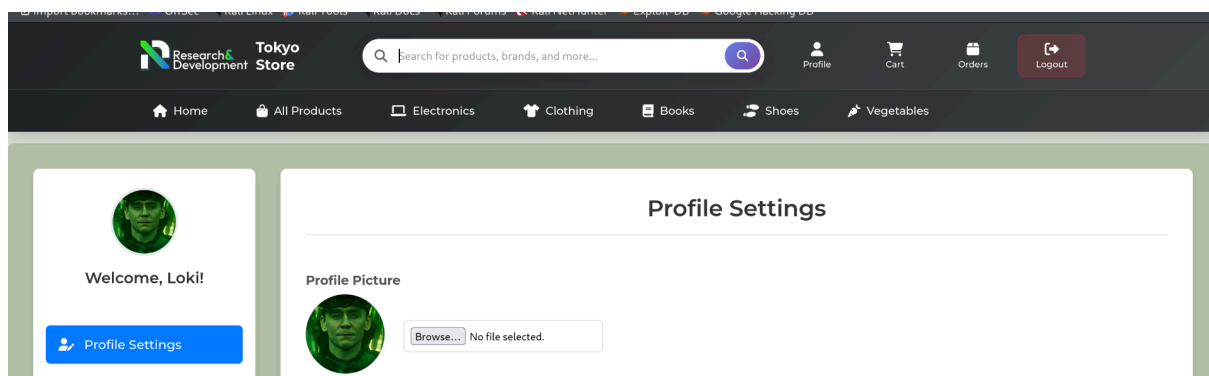
GET /dashboard HTTP/1.1
Host: ts.offenso.local
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Cookie: jwt=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImlRob3IiLCJleHAiOiE3ODZlMzgyNjd9.NfNKIGWkBLAhyFxDkW0-IfIL9YULFkUcqVAebwVvUc; session=.eJzVktF0xCAQRX9lw7PdtNBC4x_4qr4ZQ0bAXdwWEKAb1Kb_bmGzZvUPfGP0zNzc02EhwTht3UH6qE0k97uF4BzM9iAKIpK7HZmSiQ7Gwp6PPmYWoteTwrsXl-VaSavzD09Fx6CvmeipqAXT0FGqGn0zJ3_0_Ajoc-dzAocW5402ZdCqPEp29dbbdGMEj3CcAtH0BSZRwhHcF0SCdCep7T_CAeyvm4jZUX-UluzPMYjcSi1QbBDKr9hmKfd4zybKmcRP49x7-Jrs1gzyb0ErS0Jl1CXny2UKv3rn-retXQqm04r4CLruJNqzomqGKmzc6u2Z-ONoTt12Q2WGeaDE_g3BSviGaUC3VJctN0CFhkTibCABl92SCV14Vyub06Kat-chjL9sP2K4Gs6_oNN3PLQ.ai0JhA.HXUytD0ENLzv_JxU43VchcVAki0
Upgrade-Insecure-Requests: 1
Priority: u=0, i

HTTP/1.1 301 Moved Permanently
Server: nginx/1.24.0 (Ubuntu)
Date: Sat, 13 Jun 2026 07:41:19 GMT
Content-Type: text/html
Content-Length: 178
Connection: keep-alive
Location: https://ts.offenso.local/dashboard

<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
```

Attacker Session

A separate browser session was opened and authenticated as another user account.

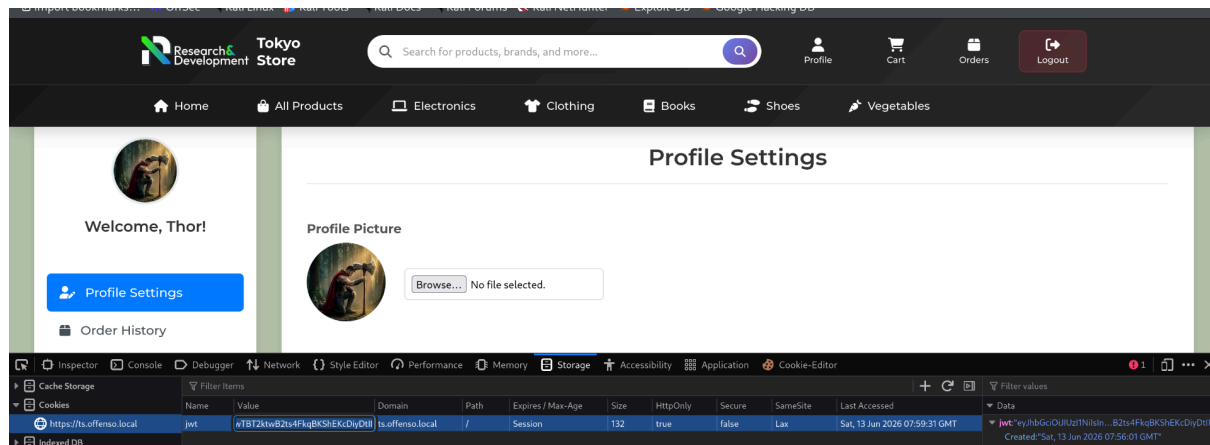


Session Hijacking

The captured JWT belonging to user Thor was manually imported into the attacker browser session.

After refreshing the page, the application accepted the stolen JWT and authenticated the attacker as Thor.

The browser displayed:



without requiring Thor's username or password.

Impact

Successful exploitation allows an attacker to:

- Hijack active user sessions.
- Access victim account information.
- Modify account settings.
- View order history and personal data.
- Perform actions on behalf of the victim.

This could result in complete account compromise without knowledge of the victim's credentials.

Recommendation

- Enforce HTTPS across the entire application.
- Configure HSTS (HTTP Strict Transport Security).
- Set the Secure attribute on all authentication cookies.
- Redirect all HTTP requests to HTTPS before processing.
- Rotate session tokens after authentication and sensitive actions.

F-08 – Reflected Cross-Site Scripting (XSS)

Severity	Medium
CVSS v3.1	6.1
Affected Endpoint	https://ts.offenso.local/subscribe

Description

The application is vulnerable to Reflected Cross-Site Scripting (XSS) in the **email** parameter of the **/subscribe** endpoint. User-supplied input is reflected back into the page without proper validation or output encoding, allowing arbitrary JavaScript execution within the victim's browser.

During testing, JavaScript code supplied through the URL parameter was executed successfully. The vulnerability also allowed HTML injection and browser redirection to external websites controlled by an attacker.

Proof of Concept

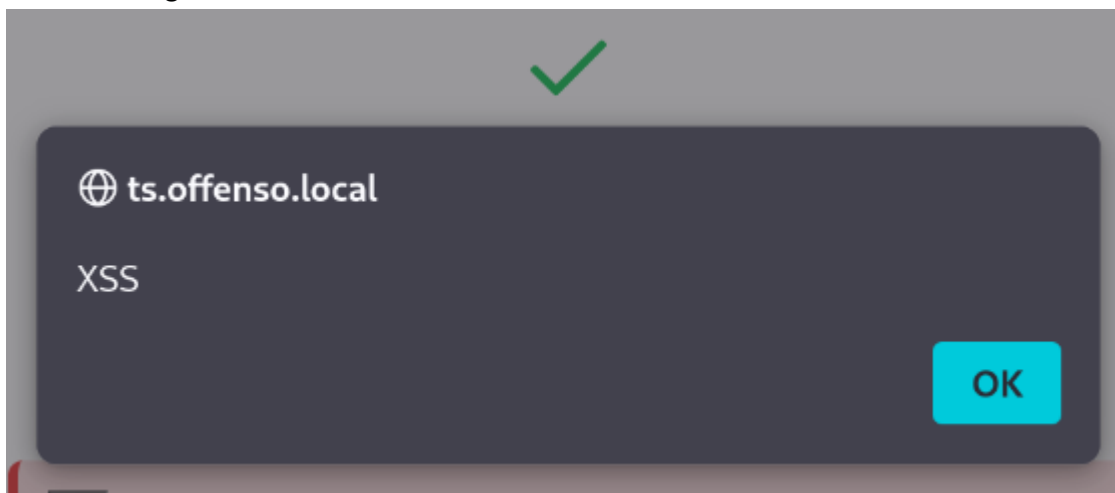
Step 1 – Basic XSS Verification

The following payload was inserted into the **email** parameter:

```
https://ts.offenso.local/subscribe?email=<script>alert('XSS')</script>
```

Result

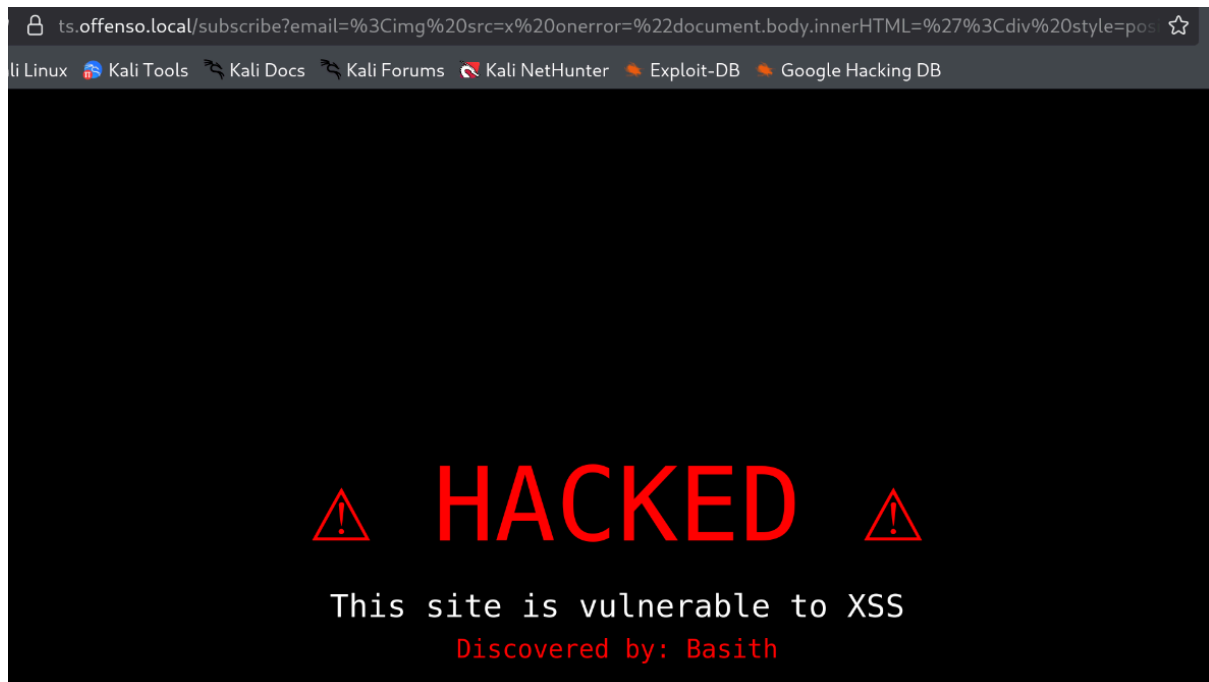
The application executed the injected JavaScript and displayed an alert box containing:



A crafted HTML payload was injected into the vulnerable parameter.

Result

The application rendered attacker-controlled HTML content and modified the appearance of the page, demonstrating that arbitrary content can be displayed to users.



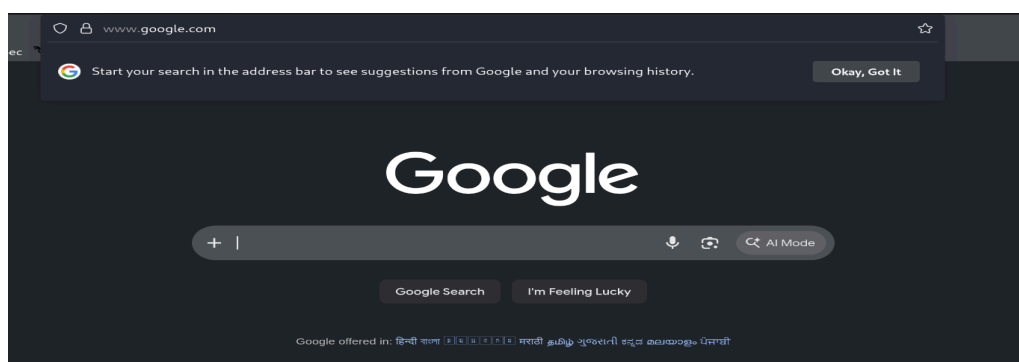
Step 3 – External Redirect via XSS

The following payload was used:

```
https://ts.offenso.local/subscribe?email=<img src=x  
onerror="window.location='https://google.com'">
```

Result

The injected JavaScript executed successfully and automatically redirected the browser to:



Vulnerable code using unsafe `innerHTML`

```
});  
  
if (response.ok) {  
    resultDiv.innerHTML = "Thanks for subscribing, " + email + "!";  
} else {  
    resultDiv.innerHTML = "Subscription failed. Please try again.";  
}  
catch (error) {
```

Impact

Successful exploitation of this vulnerability allows an attacker to:

- Execute arbitrary JavaScript in a victim's browser.
- Modify page content and perform page defacement.
- Redirect users to malicious or phishing websites.
- Display fake login forms to steal credentials.
- Access client-side data available to JavaScript.
- Perform actions on behalf of authenticated users.
- Damage user trust and application integrity.

Recommendation

- Replace `innerHTML` with `textContent` when displaying user input.
- Implement strict input validation and sanitization.
- Apply context-aware output encoding.
- Use a strong Content Security Policy (CSP).
- Reject HTML and JavaScript content in user-supplied parameters.

Conclusion

The Vulnerability Assessment and Penetration Testing (VAPT) was conducted against the target application `ts.offenso.local` to identify security weaknesses that could impact the confidentiality, integrity, and availability of the application.

During the assessment, multiple vulnerabilities were identified, including authentication weaknesses, business logic flaws, sensitive information exposure, access control issues, session management weaknesses, and Cross-Site Scripting (XSS).

These findings could allow an attacker to gain unauthorized access to user accounts, manipulate transactions, access sensitive information, hijack user sessions, and execute malicious scripts within users' browsers.

It is recommended that all identified vulnerabilities be remediated and retested to ensure the security of the application.

End of Report